

Marcador Electrónico para Natación — Frontend

Panel de control web para el marcador electrónico de competencias de natación. Este frontend se comunica en tiempo real con el firmware del microcontrolador (ESP32-S3, ver `repo marcador-electronico-para-natacion-circuitpython`) a través de un broker MQTT, y permite controlar la matriz LED, el cronómetro, la puntuación, los records y la visualización de la pantalla.

Tecnologías

- **React 19** — Framework de UI
- **Vite 7** — Build tool y dev server
- **TailwindCSS 4** — Estilos
- **MQTT.js** — Cliente MQTT vía WebSocket seguro (WSS)
- **Lucide React** — Iconos

Características

- **Control de matriz LED** — Visualización de texto y puntuación en el display de 128x64
- **Comunicación en tiempo real** — Conexión bidireccional con el ESP32-S3 vía MQTT
- **Cronómetro** — Control del tiempo de competencia (play/pausa por ronda)
- **Sistema de puntuación** — Records y scores de nadadores
- **Visualización de temperatura** — Datos del sensor LM35
- **Reloj RTC** — Hora en tiempo real (DS1307)
- **Personalización de colores** — Selector de color del texto (hex)

Requisitos

- Node.js 18+
- npm o yarn
- Un broker MQTT accesible por WebSocket (por ejemplo HiveMQ Cloud)

Configuración

1. Instalar dependencias:

```
bash npm install
```

2. Crear/editar el archivo `.env` en la raíz con:

```
env VITE_BROKER_URL=tu-broker.hivemq.cloud VITE_WS_BROKER_PORT=8884 VITE_MQTT_USERNAME=tu-usuario VITE_MQTT_PASSWORD=tu-password
```

Importante: `VITE_BROKER_URL` y `VITE_WS_BROKER_PORT` se usan para armar la URL WSS:
`wss://<VITE_BROKER_URL>:<VITE_WS_BROKER_PORT>/mqtt` .

3. Ejecutar en desarrollo:

```
bash npm run dev
```

4. Build para producción:

```
bash npm run build npm run preview # previsualizar el build
```

5. Lint:

```
bash npm run lint
```

Topics MQTT

Suscripciones (el frontend escucha)

Topic	Dirección	Descripción
esp32s3/settemp	← ESP32	Temperatura ambiente
esp32s3/user_data	← ESP32	Datos de usuarios y records
esp32s3/send_time	← ESP32	Hora RTC y temperatura
esp32s3/send_chrono	← ESP32	Estado del cronómetro
esp32s3/send_scores	← ESP32	Puntuaciones guardadas
esp32s3/send_color	← ESP32	Color configurado

Topic	Dirección	Descripción
esp32s3/screen_type	← ESP32	Tipo de pantalla activo (show / chrono)
esp32s3/send_view	← ESP32	Records visibles actualmente en pantalla

Publicaciones (el frontend envía)

Topic	Dirección	Descripción
esp32s3/get_users	→ ESP32	Solicitar datos de usuarios y records
esp32s3/get_color	→ ESP32	Solicitar configuración de color
esp32s3/settime , settemp , setcolor , new_user , chrono , del_score , del_record , screen_type	→ ESP32	Comandos de control del display (ver README del firmware)

Estructura del Proyecto

```
src/
├── components/
│   ├── MatrixDisplay.jsx    # Componente de matriz de LEDs
│   ├── TextInputForm.jsx    # Formulario de control
│   └── ColorPicker.jsx      # Selector de color
├── hooks/
│   └── use-mqtt.js          # Hook para comunicación MQTT (WSS)
├── utils/
│   └── characters.jsx        # Patrones de caracteres para matriz
├── App.jsx                  # Componente principal
├── main.jsx                  # Punto de entrada
└── index.css                 # Estilos base
```

Relación con los otros repositorios

- **Frontend** (este repo): interfaz web de control.
- **CircuitPython** (`marcador-electronico-para-natacion-circuitpython`): firmware que maneja la matriz LED, el cronómetro y el RTC.
- **General** (`marcador-electronico-para-natacion`): prototipo inicial del firmware.

MIT